

Architecture Decision Records

Low Accuracy Kinda Intelligent Tracking Unit (LAKITU)

Índice

ADR	Decisión
ADR-001	Adopción de una arquitectura basada en capas.
ADR-002	Tracking a prueba de obstrucciones con inferencia de entrada y salida.
ADR-003	Envío de datos únicamente en los cambios de estado.
ADR-004	Actualizaciones basadas en eventos con estado completo.
ADR-005	Uso de AWS IoT Core y SQS.
ADR-006	Componente de autenticación personalizado.
ADR-007	Uso de Redis caché.
ADR-008	Uso de PostgreSQL y Amazon RDS.
ADR-009	Uso de Amazon S3.
ADR-010	Comunicación dual: WebSocket y REST.
ADR-011	Uso de una instancia EC2 dedicada en el backend cloud.
ADR-012	Uso de Next.js hosteado en Vercel para el frontend.
ADR-013	Uso de Python y FastAPI para el backend cloud.
ADR-014	Uso de SQLAlchemy Core con psycopg3 para gestionar PostgreSQL.
ADR-015	Uso de PyJWT y passlib con bcrypt para autenticación.
ADR-016	[vacío]
ADR-017	[vacío]
ADR-018	[vacío]
ADR-019	[vacío]
ADR-020	[vacío]

ADR-001

Título

Definición de un patrón de arquitectura a seguir.

Fecha

24/03/2026

Estado

Tomada

Contexto

El sistema debe cumplir múltiples responsabilidades mediante el uso de tecnologías diferentes: computer vision en hardware limitado, servicios cloud para el backend, entrega de datos en tiempo real, almacenamiento persistente, y un frontend web. Estas son claramente diferentes, y tienen diferentes necesidades de escalabilidad, despliegue, y mantenimiento. Un patrón de arquitectura con estructura definida era necesario para organizar estas responsabilidades de una manera entendible y viable dado el alcance del proyecto.

Decisión

Se adoptará una **arquitectura en capas**, organizada en cinco capas: Presentación, Ingesta de datos, Lógica de negocio, Persistencia de datos, y Datos.

Factores decisivos

1. El equipo es pequeño y el proyecto es académico, por lo que un patrón simple, fácil de entender y de gestionar es favorable.
2. El sistema tiene una separación vertical clara de responsabilidades: entrada de datos al sistema (o a partes del sistema), lógica, almacenamiento y persistencia, y visualización (frontend).
3. La arquitectura en capas está bien documentada y es altamente usada en la industria; y todos los integrantes ya hemos trabajado con la misma, por lo que la curva de aprendizaje es inexistente.
4. El patrón permite a cada capa ser desarrollada, testeada, y modificada de manera casi independiente, útil dada la naturaleza iterativa que puede tener el proyecto.

Alternativas

- **Microservicios:** Ofrece escalabilidad y despliegue totalmente independiente entre servicios, pero introduce una complejidad operacional desproporcionada para el alcance de un proyecto universitario de esta escala.
- **Arquitectura Event-Driven:** Puede ayudar a desacoplar componentes y a procesar la información de forma asíncrona, pero esto ya es posible con otras arquitecturas, y el uso de una EDA dificultaría el razonamiento sobre estados y flujo a través del sistema en general. La comunicación basada en eventos será usada por sus beneficios, pero no centraremos la arquitectura en torno a esta.
- **Monolito:** Poco óptimo e imposible dados los requerimientos.

Pros y contras de una arquitectura en capas

Pros:

- Separación intuitiva de responsabilidades que además se vincula tanto con la separación lógica como física necesaria del sistema.
- Facilita separar el trabajo entre los integrantes del equipo.
- Entendido y conocido en la industria y por usuarios académicos.

Contras:

- Puede introducir acoplamiento si se traspasan capas o si se definen de mala manera.
- La transmisión de información a través de múltiples capas puede afectar negativamente el rendimiento, aunque a esta escala dicho efecto es despreciable.

Consecuencias

Cada capa debe respetar las reglas de comunicación impuestas por el patrón (la capa n solo puede interactuar con las capas n-1 y n+1).

Comentarios

El uso de una arquitectura en capas no inhibe el uso de patrones locales. El patrón en capas describe la estructura a nivel macroscópico, cada capa puede usar sus propios patrones para su estructura interna.

ADR-002

Título

Tracking a prueba de obstrucciones mediante inferencia de entrada y salida.

Fecha

16/03/2026

Estado

Tomada

Contexto

La cámara está montada en un ángulo elevado, pero no completamente cenital, lo cual significa que autos estacionados cerca de la cámara pueden bloquear la visión de los lugares más lejanos. Una detección estándar basada en la imagen individual de cada frame fallaría al no poder observar estos lugares. El sistema necesita una forma de poder determinar con cierta certeza si los lugares cuya visibilidad está obstruida están ocupados o libres.

Decisión

Implementar un sistema de seguimiento de los autos que sea capaz de inferir cuando uno de ellos entra o sale de un lugar, aunque no tenga visibilidad directa del auto en dicho lugar. Este sistema se basará en zonas de entrada/salida por lugar, y un modelo entrenado de machine learning.

Factores decisivos

1. El ángulo de la cámara causa que la detección basada en frames estáticos falle o sea poco confiable para lugares cuya visión esté obstruida.
2. Existen soluciones livianas, como librerías, conocidas y de buen funcionamiento que son viables para su uso en un Raspberry Pi.

Alternativas

- **Detección basada en frames estáticos:** Directamente clasificar cada lugar como ocupado o libre es más simple, pero deja de funcionar en las situaciones mencionadas. Descartado porque simplemente no permite cumplir los requerimientos con una precisión aceptable dado el ángulo de la cámara disponible.

- **Múltiples cámaras:** Agregar una segunda cámara para eliminar los puntos ciegos. Descartado dada la carga extra en complejidad y en hardware. Excede el alcance del proyecto.
- **Reposicionamiento de la cámara:** Tener la cámara en un punto más alto o más físicamente arriba del centro del estacionamiento solucionaría el problema, pero simplemente no es posible dada la estructura del mismo.

Pros y contras del tracking con tolerancia a obstrucciones

Pros:

- Soluciona la limitación real del ángulo y posición de la cámara sin necesitar cambios o agregados en el hardware.
- Produce más datos útiles como efecto secundario (movimiento de los autos, tiempo en estacionar, etc).
- Más robusto que la detección estática.

Contras:

- Significativamente más complejo de implementar y ajustar.
 - Errores de trackeo pueden hacer que el sistema se desincronice de la realidad con el tiempo; un mecanismo de sincronización puede llegar a ser necesario.
 - Computacionalmente más demandante para el Raspberry Pi.
-
-

ADR-003

Título

Cuándo comunicar al backend el estado actual del estacionamiento.

Fecha

23/03/2026

Estado

Tomada

Contexto

El backend del sistema debe conocer el estado del estacionamiento en tiempo real constantemente, y para esto es necesario decidir qué tipo de información (ADR-004), y con qué frecuencia, enviará el Raspberry Pi a AWS.

Decisión

El Raspberry Pi tendrá un estado interno del estacionamiento, el cual será enviado al backend solo cuando se tenga certeza de que el mismo cambie, ya sea por un auto estacionando, o yéndose de su lugar.

Factores decisivos

1. Enviar el estado del estacionamiento de forma continua o a intervalos fijos generaría tráfico de red completamente redundante en la gran mayoría de los casos, ya que el estacionamiento permanece sin cambios durante períodos prolongados. Transmitir solo ante cambios reales reduce drásticamente el volumen de mensajes enviados a AWS IoT Core y, en consecuencia, el costo operativo y el consumo de ancho de banda del Pi.
2. La naturaleza del sistema es inherentemente orientada a eventos: lo que realmente le importa al backend es conocer los cambios en el estado tan pronto como sucedan. Un modelo de transmisión basado en eventos se alinea directamente con este objetivo, simplificando tanto la lógica del Pi como la del Cloud Receptor.
3. Reducir la cantidad de mensajes enviados disminuye la carga de procesamiento en el sistema en general, evitando escrituras innecesarias en la base de datos y actualizaciones redundantes.

Alternativas

- **Envío periódico a intervalos fijos (heartbeat):** El Pi enviaría el estado completo del estacionamiento cada N segundos, independientemente de si hubo cambios. Descartado porque genera tráfico, costo y carga de procesamiento innecesarios cuando el estacionamiento está estático, que es la mayor parte del tiempo.
- **Streaming continuo de video o datos de visión:** Transmitir los resultados del procesamiento de visión computacional de forma continua al backend. Descartado por las mismas razones en cuanto a la carga de red y cómputo del Raspberry Pi, y completamente innecesario dado que el procesamiento de visión ocurre localmente en el dispositivo.

Pros y contras del envío solo ante cambios de estado

Pros:

- Minimiza el tráfico de red y el costo asociado al uso de servicios de AWS.
- Reduce la carga de escritura en PostgreSQL y Redis, ya que solo se persisten y cachean eventos con información nueva.
- Simplifica el modelo mental del sistema: cada mensaje recibido por el backend representa un cambio real y significativo en el estacionamiento.
- Extiende la vida útil del hardware y reduce el consumo energético del Pi al minimizar la actividad de red.

Contras:

- El backend no recibe confirmaciones periódicas de que el Pi sigue operativo; un fallo silencioso del Pi podría pasar desapercibido durante un período prolongado si no se implementa un mecanismo de heartbeat separado para monitoreo de conectividad.
- Si el sistema de tracking del Pi incurre en un error y no detecta un cambio de estado real, el backend permanecerá desactualizado hasta el próximo evento detectado, sin mecanismo automático de corrección a corto plazo.

Consecuencias

Dado que el backend no recibirá mensajes continuos del Pi, debe implementarse un mecanismo complementario de detección de conectividad (por ejemplo, un heartbeat periódico liviano o el monitoreo del estado de conexión MQTT en IoT Core) para alertar si el dispositivo deja de responder. Adicionalmente, al reconectarse tras una desconexión, el Pi debe re-emitir el estado actual completo del estacionamiento para garantizar que el backend quede sincronizado.

Comentarios

Esta decisión se complementa directamente con ADR-012 (envío del estado completo en cada evento): dado que cada mensaje ya contiene el estado íntegro del estacionamiento, la pérdida ocasional de un mensaje no produce inconsistencias acumulativas, lo que hace

que el modelo de envío basado en cambios sea aún más robusto de lo que sería con un esquema de deltas.

ADR-004

Título

Cómo comunicar al backend el estado actual del estacionamiento.

Fecha

16/03/2026

Estado

Tomada

Contexto

Cuando un lugar de estacionamiento cambia su ocupación, el sistema debe comunicar el nuevo estado al cloud. Se requiere decidir si el Pi enviará todo el estado del estacionamiento, o solo las partes que hayan cambiado.

Decisión

En cada evento de cambio de ocupación, el **Raspberry Pi envía el estado completo actual** del estacionamiento entero (todos los lugares y sus estados) como un único payload JSON, en lugar de solo los lugares que cambiaron.

Factores decisivos

1. El estacionamiento monitoreado es pequeño, lo que significa que el payload del estado completo es pequeño y el costo de ancho de banda de enviar el estado completo es despreciable.
2. Enviar el estado completo elimina el riesgo de que el modelo del cloud diverja del modelo del Pi debido a mensajes perdidos o fuera de orden. El estado del cloud simplemente se sobrescribe con el estado del Pi en cada evento.
3. La gestión de actualizaciones constantes a partes específicas del estado, mientras se mantiene el resto intacto involucra una complejidad muy superior, sobre todo en períodos pico (como las 8am) cuando pueden detectarse varios cambios por minuto. La lógica del cloud receptor se ve significativamente simplificada.

Alternativas

- **Solo actualizaciones de variaciones:** Enviar solo el/los lugar(es) que cambiaron por evento. Más eficiente en ancho de banda, pero introduce riesgo de consistencia de estado y requiere que el cloud mantenga un modelo de estado completo que pueda

actualizarse incrementalmente. Para un estacionamiento pequeño, el ahorro de ancho de banda es despreciable.

- **Actualizaciones de variaciones con sincronización periódica del estado completo:** Enfoque híbrido. Más complejo sin beneficio significativo a esta escala.

Pros y contras del payload de estado completo por evento

Pros:

- Garantiza consistencia eventual: incluso si mensajes anteriores se perdieron, el siguiente evento resincroniza completamente el estado del cloud.
- Simplifica significativamente el Cloud Receptor y la lógica de actualización de estado actual.
- Sencillo de depurar: cada mensaje es autocontenido y completamente interpretable sin historial.

Contras:

- Tamaño de mensaje ligeramente mayor por evento (aunque despreciable para un estacionamiento pequeño).
- La base de datos almacena datos redundantes (estados de lugares sin cambios) con cada registro de evento, incrementando ligeramente el uso de almacenamiento.
- El evento almacenado en la base de datos representa un snapshot completo del estacionamiento en un punto en el tiempo, no solo un cambio. Esto hace que las consultas históricas sean más simples (cada fila es un estado completo).

ADR-005

Título

Selección de un medio para la comunicación entre el Raspberry Pi y el servidor cloud.

Fecha

24/03/2026

Estado

Tomada

Contexto

El Raspberry Pi debe enviar de forma confiable los eventos de cambio de estado del estacionamiento al backend cloud. Estos son payloads JSON pequeños y generalmente infrecuentes, disparados por cambios de estado. El canal de comunicación debe manejar: conectividad intermitente, autenticación segura del dispositivo, ingesta de mensajes escalable, y la capacidad de que el servidor cloud envíe comandos de vuelta al Pi (sacar una foto a demanda).

Decisión

Usar **AWS IoT Core** como proveedor de comunicación mediante MQTT para conectar el Pi a cloud y cloud al Pi. Una **IoT Core Rule** reenvía los mensajes entrantes del Pi a una **cola SQS**, que el componente Cloud Receptor en EC2 consume. Para las solicitudes de fotos bajo demanda, el Cloud Receptor publica directamente en un topic de IoT Core al que el Pi está suscripto.

Factores decisivos

1. IoT Core soporta nativamente MQTT, que es liviano y está bien adaptado para hardware embebido como el Raspberry Pi operando sobre redes potencialmente limitadas.
2. IoT Core gestiona la autenticación y autorización del dispositivo mediante certificados X.509, eliminando la necesidad de gestionar credenciales.
3. SQS provee una cola de mensajes confiable y ordenada con entrega garantizada, desacoplando la tasa de publicación del Pi de la tasa de consumo del Cloud Receptor.
4. El Rule Engine de IoT Core permite enrutar mensajes a SQS (y potencialmente otros servicios) sin código personalizado, simplificando la programación de la ingesta.

5. La naturaleza bidireccional de MQTT (los topics pueden ser tanto publicados como suscritos) soporta nativamente tanto el envío de eventos como la recepción de comandos en una única conexión.

Alternativas

- **HTTPS directo al backend:** El Pi podría enviar solicitudes POST directamente a un endpoint REST en el Cloud Receptor. Descartado porque requiere que parte del backend esté expuesta públicamente, carece de gestión de identidad de dispositivos integrada, y es carente ante falta de disponibilidad temporal del lado cloud.
- **AWS Kinesis Data Streams:** Diseñado para streams de datos de alto volumen y continuos. Excesivo para un sistema que genera eventos discretos e infrecuentes, y añade costo y complejidad innecesarios.
- **WebSockets directo al backend:** Soportaría comunicación bidireccional pero requiere mantener conexiones persistentes, manejar lógica de reconexión y de autenticación del dispositivo, y es operacionalmente más pesado que MQTT para un dispositivo embebido.

Pros y contras de IoT Core + SQS

Pros:

- Comunicación MQTT ya gestionada y escalable sin servidor ni servicio que mantener.
- La autenticación de dispositivos mediante certificados X.509 es segura y está desacoplada del resto de la aplicación.
- SQS funciona como buffer de eventos si el Cloud Receptor está temporalmente caído o lento, asegurando que no se pierdan eventos.
- Los topics MQTT bidireccionales sirven tanto para el pipeline de eventos como para el flujo de comandos de foto bajo demanda.
- El Rule Engine de IoT Core desacopla el enrutamiento de mensajes de la lógica de la aplicación.

Contras:

- Introduce dependencias específicas de AWS; migrar fuera del mismo sería muy complejo.
 - Agrega latencia respecto a una conexión directa.
-
-

ADR-006

Título

Cómo autenticar a los usuarios.

Fecha

24/03/2026

Estado

Tomada

Contexto

El sistema requiere autenticación de usuarios para proteger el backend y sus funcionalidades. Los usuarios deben iniciar sesión para ver los datos del estacionamiento y, en el caso de usuarios con privilegios, para comandar capturas de fotos. Se debe elegir una solución de autenticación: ya sea un servicio gestionado de AWS como Cognito, o un componente de autenticación personalizado integrado en el backend de la aplicación.

Decisión

Implementar un **componente de autenticación personalizado** dentro de la capa de Lógica de Negocio en el backend cloud. Este componente gestiona el registro de usuarios, el inicio de sesión, la emisión y validación de JWT, y la gestión de sesiones.

Factores decisivos

1. Los requisitos de autenticación son simples. El extenso conjunto de funcionalidades de Cognito quedaría en su mayoría sin uso.
2. Un componente de autenticación personalizado otorga al equipo control total sobre el formato del token, la lógica de validación y el modelo de datos de usuario.
3. Construir la autenticación desde cero es un ejercicio de aprendizaje valioso dentro del contexto académico del proyecto.
4. La cantidad de configuración de Cognito (user pools, app clients, hosted UI, callback URLs) no es trivial y consumiría tiempo de desarrollo mejor invertido en funcionalidades principales.

Alternativas

- **AWS Cognito**: Servicio de identidad completamente gestionado con soporte para MFA, OAuth e inicio de sesión social. Descartado porque introduce considerable

complejidad de configuración, es un coste extra en la cuota de AWS, y sus servicios funcionales son desproporcionados para las necesidades del sistema.

- **Auth0 / proveedor de terceros:** Razones similares a Cognito. Dependencia externa, posible costo tras el nivel gratuito, y más opciones de integración de la necesaria.
- **Sin autenticación:** Dejar el sistema abierto. Incluso en una escala reducida, un sistema de estacionamiento desprotegido que expone datos de cámara en vivo y permite solicitudes de fotos no es aceptable.

Pros y contras del componente de autenticación personalizado

Pros:

- Control total sobre la estructura del token, las políticas de expiración y el modelo de usuario.
- Sin dependencia de servicio externo para la autenticación; el sistema permanece autocontenido.
- Más simple de entender y debugear para todo el equipo.

Contras:

- El equipo es responsable de la corrección de la seguridad (hashing de contraseñas, firma de tokens, manejo de expiración). Los errores pueden introducir vulnerabilidades.
- Sin soporte integrado para funcionalidades avanzadas como MFA o inicio de sesión social si los requisitos cambian.

Comentarios

El componente de autenticación personalizado debe usar librerías bien establecidas para el hashing de contraseñas (como **bcrypt**) y el manejo de JWT.

ADR-007

Título

Gestión óptima del estado del estacionamiento en tiempo real.

Fecha

24/03/2026

Estado

Tomada

Contexto

El frontend web necesita mostrar el estado actual del estacionamiento en tiempo casi real. Cada evento de cambio de estado desde el Pi actualiza el estado completo del estacionamiento. Este estado es consultado frecuentemente por los clientes conectados y debe servirse con latencia muy baja. Consultar la base de datos en cada lectura sería lento y generaría carga innecesaria en la misma.

Decisión

Usar **Redis** como caché en memoria dentro de la capa de Lógica de Negocio para almacenar el estado actual del estacionamiento. Cuando llega un nuevo evento desde el Pi, el Cloud Receptor actualiza Redis con el nuevo estado. El Cloud Backend lee de Redis para responder las consultas de los clientes sobre el estado actual.

Factores decisivos

1. Redis está diseñado específicamente para acceso de baja latencia a pares clave-valor, haciéndolo ideal para servir el estado actual (una única estructura de datos) a muchos clientes concurrentes.
2. Separar la preocupación del "estado actual" de la base de datos persistente reduce la carga sobre esta y mantiene el camino de lectura simple y rápido.
3. El estado actual del estacionamiento es un payload pequeño; Redis está perfectamente dimensionado para este caso de uso.

Alternativas

- **Consultar la base de datos directamente para el estado actual:** Simple, pero introduce latencia y carga de base de datos en cada lectura. A medida que crece el número de clientes conectados, esto se convierte en un cuello de botella.

- **Estado en memoria de Cloud Backend:** Almacenar el estado actual en la memoria de la aplicación. Rápido, pero no durable ante reinicios, y no es compartible con tener múltiples instancias del proceso (por si se deseara escalar horizontalmente en un futuro).

Pros y contras de Redis como caché de estado

Pros:

- Latencia de lectura en milisegundos para consultas del estado actual.
- Reduce la carga sobre la base de datos significativamente.
- Modelo de datos simple (el estado del estacionamiento puede almacenarse como un hash de Redis o string JSON).
- Librerías cliente bien soportadas en todos los lenguajes de backend principales.

Contras:

- Agrega un componente de infraestructura adicional para desplegar y gestionar.
- Los datos de Redis se pierden al reiniciar a menos que se configure persistencia; para el caché de estado actual esto es tolerable ya que el Pi re-emitirá el estado actual al reconectarse.
- Introduce una posibilidad de inconsistencia entre el caché de Redis y la base de datos si las actualizaciones no se manejan bien.

Consecuencias

El camino de escritura para un nuevo evento de estacionamiento debe actualizar Redis primero (para servicio inmediato a los clientes) y luego almacenarse en la base de datos de forma asíncrona.

Comentarios

Redis se ubica en la capa de Lógica de Negocio en lugar de la capa de Datos porque sirve como caché operacional gestionado y utilizado únicamente por un componente para cumplir con su lógica de negocio, no como almacenamiento durable. Esta distinción se refleja en el diagrama de arquitectura.

ADR-008

Título

Decisión de la base de datos persistente principal.

Fecha

24/04/2026

Estado

Tomada

Contexto

El sistema debe persistir eventos de estacionamiento (cambios de estado), cuentas de usuario y datos históricos de ocupación para consultas a través de la API REST. Se debe elegir una tecnología de base de datos que sirva como almacenamiento persistente del sistema.

Decisión

Usar **PostgreSQL** gestionado por AWS a través de RDS como base de datos relacional para todos los datos estructurados persistentes.

Factores decisivos

- Los datos del sistema son relacionales: los eventos referencian lugares, los lugares pertenecen a estacionamientos, los usuarios tienen roles, etc. Un modelo relacional es una opción natural.
- PostgreSQL es muy usado, está bien documentado y todo el equipo ya ha trabajado con él.
- Las consultas históricas complejas (por ejemplo, "ocupación promedio por lugar en los últimos 7 días" para calcular estadísticas) son sencillas a través de SQL sin necesidad de capas de consulta adicionales.
- RDS facilita el despliegue y la gestión de la base de datos significativamente.

Alternativas

- DynamoDB**: NoSQL gestionado de AWS con escalado serverless. Descartado porque el modelo de datos es relacional, y los patrones de acceso de DynamoDB son menos intuitivos para consultas históricas.
- MongoDB**: Almacén de documentos que ofrece flexibilidad de esquema. Descartado porque el esquema está bien definido y es relacional; la flexibilidad de un almacén de

documentos no es necesaria para este tipo de datos y su falta de relaciones forzadas reduce las garantías de integridad de datos.

- **SQLite:** Base de datos relacional embebida y liviana. Descartado porque no soporta escrituras concurrentes desde múltiples hilos/procesos de la aplicación en EC2.

Pros y contras de PostgreSQL

Pros:

- Cumplimiento ACID completo asegura la integridad de los datos para los registros de eventos.
- Capacidades de consulta SQL para análisis histórico.
- Familiar para el equipo.

Contras:

- No escala horizontalmente sin configuración adicional significativa, aunque esto no es una preocupación a la escala del proyecto.

ADR-009

Título

Selección de un medio para almacenar las fotos de eventos de estacionamiento.

Fecha

24/03/2026

Estado

Tomada

Contexto

El sistema captura fotos del estacionamiento asociadas con cada evento de cambio de estado. Estas fotos deben almacenarse de forma persistente y estar accesibles para revisión histórica. Almacenar datos binarios en PostgreSQL es posible pero poco escalable. Se necesita una solución de almacenamiento separada para las imágenes.

Decisión

Almacenar todas las fotos de eventos de estacionamiento en **Amazon S3**, organizado como un data lake. Cada foto se almacena con una clave que referencia el ID del evento correspondiente en PostgreSQL (`events/{event_id}/photo.jpg`). El componente **Persistencia** gestiona todas las interacciones con S3.

Factores decisivos

1. S3 está diseñado específicamente para almacenamiento de objetos y es mucho más rentable y eficiente para estos que almacenar arrays binarios en PostgreSQL.
2. S3 provee 11 niveles de durabilidad, lo cual es más que apropiado para registros fotográficos históricos.
3. Las políticas de ciclo de vida de S3 pueden usarse para archivar o eliminar automáticamente fotos antiguas, gestionando los costos de almacenamiento a largo plazo.

Alternativas

- **Almacenar fotos en PostgreSQL como binarios:** Simple desde la perspectiva de desarrollo, pero incrementa el tamaño de la DB exponencialmente, degrada el rendimiento de las consultas, y no es la forma en que se deben usar las bases de datos relacionales.

- **Almacenar fotos en el volumen EBS de EC2:** Atado a una única instancia, no persistente ante reemplazos de instancia, y no escala independientemente del backend de lógica. Descartado.
- **Almacenamiento de terceros (por ejemplo, Cloudinary):** Agrega dependencia externa y costo. S3 ya está en el ecosistema de AWS que se usa para el resto del backend.

Pros y contras de S3 como almacenamiento de fotos

Pros:

- Almacenamiento de objetos rentable y altamente durable.
- Desacoplado de la base de datos y del backend de lógica, escalando independientemente.

Contras:

- Agrega dependencia de la disponibilidad de S3 para la recuperación de fotos.

Consecuencias

El componente Persistencia debe implementar la siguiente lógica de escritura: primero escribir la foto en S3, luego insertar el registro del evento en PostgreSQL con la clave de S3. Las fallas parciales (foto subida pero falla la inserción en DB) deben manejarse con un mecanismo de limpieza o reintento.

ADR-010

Título

Definición de el/los medio(s) de comunicación entre el sistema y los usuarios finales.

Fecha

16/03/2026

Estado

Tomada

Contexto

El frontend tiene dos patrones de acceso a datos claramente distintos: (1) consumir el estado actual y en vivo del estacionamiento a medida que cambia en tiempo real, y (2) consultar datos históricos de ocupación. Estos dos tipos de peticiones tienen características fundamentalmente diferentes en términos de latencia, frecuencia de transmisión, duración de la conexión y volumen de datos.

Decisión

Exponer **dos APIs separadas desde el backend**: una **API WebSocket** para actualizaciones del estado del estacionamiento en tiempo real y solicitudes de fotos bajo demanda, y una **API REST** para consultas de eventos históricos y operaciones administrativas. Ambas APIs comparten el mismo componente Auth para la autenticación.

Factores decisivos

1. La consulta del estado actual del estacionamiento está centrado en la idea de notificaciones (no push) en tiempo real: el backend debe avisar a todos los clientes conectados inmediatamente cuando ocurre un cambio de estado (comunicación servidor a cliente). El otro tipo de consulta en tiempo real es el de sacara fotos a demanda, comunicación que debe ocurrir en el sentido opuesto (cliente a servidor). WebSocket es el protocolo natural para estos requisitos.
2. Las consultas históricas son operaciones de petición y respuesta sin requisito de tiempo real. REST está bien adaptado, es ampliamente entendido y usado, fácil de documentar y testear.
3. Mezclar semánticas de tiempo real y request-response en un único protocolo, ya sea implementar consultas históricas con WebSockets o comunicación en tiempo real con REST, complicaría innecesariamente tanto la implementación del cliente como la del servidor.

4. Separar las APIs permite que cada una sea diseñada y optimizada para su propio tipo de acceso.

Alternativas

- **Solo WebSocket (todas las consultas sobre WebSocket):** Técnicamente viable, pero fuerza un patrón request-response sobre una conexión bidireccional, requiriendo un incrementando significativo de la complejidad del cliente.
- **Solo REST con consultas periódicas:** El cliente consulta periódicamente un endpoint REST para el estado actual. Introduce latencia, desperdicia mucho el ancho de banda, y es una experiencia de usuario inferior comparada con server-push vía WebSocket.
- **Eventos para tiempo real, REST para histórico:** Es más simple que WebSocket para push unidireccional, pero no soporta la comunicación bidireccional necesaria para las solicitudes de fotos a demanda. Dos canales serían necesarios, haciendo el desarrollo más complejo.
- **WebSocket para tiempo real, GraphQL para histórico:** Opción muy atractiva, GraphQL simplifica enormemente las consultas a datos relacionales estructurados, sin embargo sus beneficios son más aparentes en sistemas con bases de datos más complejas que la que trataremos nosotros. Descartado por aportar beneficio limitado para un sistema con pocos tipos de datos, como el nuestro, y por la ligeramente superior complejidad de definición y documentación que puede agregar.

Pros y contras de la estrategia dual WebSocket + REST

Pros:

- Cada API está diseñada exactamente para su patrón de acceso, manteniendo ambas implementaciones limpias y centradas en su responsabilidad.
- La API REST es trivial de testear con herramientas estándar (como Postman) y documentar.
- El ciclo de vida de la conexión WebSocket (conectar, recibir pushes, enviar comandos) está bien definido y es simple de implementar para el frontend.
- El componente Auth compartido evita duplicar la lógica de autenticación.

Contras:

- El cliente del frontend debe gestionar dos tipos de conexión diferentes.
- Ambas APIs deben mantenerse y versionarse independientemente.

Consecuencias

El Cloud Backend debe implementar un registro de conexiones WebSocket para rastrear todos los clientes actualmente conectados y transmitir eventos de cambio de estado a todos ellos cuando llega un nuevo evento desde el Pi. La API REST debe definir semánticas claras de paginación y filtrado para las consultas de eventos históricos, y en general seguir buenas prácticas típicas.

ADR-011

Título

Despliegue del backend.

Fecha

24/03/2026

Estado

Tomada

Contexto

Los componentes del backend cloud (Cloud Receptor, Cloud Backend, Auth, Persistencia) deben ejecutarse en algún lugar de AWS. La arquitectura soporta conexiones WebSocket persistentes para actualizaciones en tiempo real, un consumidor de SQS, y procesos que mantienen estado en memoria vinculado a Redis. Se debe elegir una plataforma de cómputo.

Decisión

Ejecutar todos los componentes del backend cloud en una **única instancia EC2** (t3.small o t3.medium), con componentes desplegados como procesos separados o contenedores Docker en el mismo host.

Factores decisivos

1. Las conexiones WebSocket persistentes requieren un proceso servidor de larga ejecución, lo cual es incompatible con el modelo de ejecución serverless y de corta duración de AWS Lambda.
2. El consumidor de SQS (Cloud Receptor) es un loop que también requiere un proceso persistente.
3. Una única instancia EC2 es operacionalmente simple para un equipo pequeño, evitando la sobrecarga de orquestar múltiples servicios.

Alternativas

- **AWS Lambda (serverless):** Rentable y escala automáticamente, pero fundamentalmente incompatible con conexiones WebSocket persistentes y loops consumidores de SQS con estado. Requeriría la integración WebSocket de AWS API Gateway, añadiendo complejidad innecesaria.

- **AWS Elastic Beanstalk:** Despliegue basado en EC2 gestionado. Más fácil que EC2 crudo, pero introduce abstracción adicional que puede ser más difícil de depurar. EC2 crudo da al equipo más control directo y valor de aprendizaje.
- **Múltiples EC2, separando por servicio:** Permite desacoplar físicamente los componentes, pero agrega complejidad operacional y de desarrollo excesiva. La gestión de múltiples máquinas, con sus diferentes interfaces de comunicación y la latencia que esta comunicación agregaría son demasiadas desventajas a cambio de poca ganancia.

Pros y contras de una única instancia EC2

Pros:

- Simple de usar, acceder por SSH y gestionar.
- Soporta todos los patrones requeridos (procesos persistentes, loops de petición a SQS, comunicación WebSocket).
- Nos permite aprender sobre administración de servidores Linux.

Contras:

- Punto único de falla; si la instancia cae, todos los componentes del backend quedan no disponibles.
 - Sin escalado automático; requeriría intervención manual ante carga inesperada.
-
-

ADR-012

Título

Despliegue del frontend.

Fecha

16/03/2026

Estado

Tomada

Contexto

El sistema requiere un frontend web para que los usuarios visualicen la disponibilidad del estacionamiento en tiempo real y los datos históricos. El frontend se comunicará con el backend mediante REST (consultas históricas) y WebSocket (actualizaciones en vivo). Se debe seleccionar un framework para desarrollarlo y una plataforma para hostearlo.

Decisión

Construir el frontend como una aplicación **Next.js** hosteada en los servidores de **Vercel**.

Factores decisivos

1. Next.js provee una experiencia de desarrollo muy cómoda con enrutamiento basado en carpetas, y capacidades de server-side rendering.
2. El modelo de despliegue de Vercel simplifica significativamente el pipeline de CI/CD del frontend.
3. El nivel gratuito de Vercel es suficiente para las necesidades de tráfico del proyecto y elimina el costo de hosting para el frontend.
4. Next.js es un framework ampliamente usado con documentación extensa, y parte del equipo ya es familiar con este, haciendo que la incorporación al proyecto sea directa.

Alternativas

- **React simple (Vite/CRA) en AWS:** Mayor control pero requiere configuración adicional de AWS y complejiza ligeramente el desarrollo en comparación a next.js; todo sin agregar ventajas significativas sobre el hosteo en Vercel.
- **Vue.js / Angular:** Alternativas válidas, pero el equipo tiene mayor familiaridad con React/Next.js.

- **Frontend sin framework, puro html y js:** Se puede argumentar que otorga más flexibilidad, pero la complejidad extra al proceso de desarrollo no vale nada la pena. Descartado.

Pros y contras de Next.js en Vercel

Pros:

- Despliegues sin configuración directamente desde Git.
- Despliegues de "preview" a través de pull requests simplifican la revisión de código y el CI/CD.
- Soporte integrado para variables de entorno, rutas de API y edge middleware.
- Sin costo de hosteo del frontend.

Contras:

- Dependencia de Vercel para el frontend web.
- Las conexiones WebSocket serán un poco más complejas de configurar.

Consecuencias

Las conexiones WebSocket deben apuntar directamente al backend en EC2, ya que el modelo serverless de Vercel no soporta conexiones persistentes al servidor. Adicionalmente, será necesario crear un certificado TLS desde el lado del backend, ya que los navegadores no aceptan una conexión WebSocket sin certificado. Para esto seguramente se use Nginx con Let's Encrypt via Certbot.

ADR-013

Título

Lenguaje y framework principales para el desarrollo del backend cloud.

Fecha

11/04/2026

Estado

Tomada

Contexto

El backend debe ser capaz de establecer una comunicación con los clientes a través del protocolo WebSocket, así como ofrecer endpoints REST. A su vez es necesario elegir un lenguaje que permita fácilmente interactuar con una base de datos PostgreSQL, con S3, Redis, SQS e IoT Core en AWS, y que ofrezca servicios de autenticación conocidos para una sencilla implementación del componente de autenticación personalizado.

Decisión tomada

Construir el backend usando **Python**, particularmente con el framework **FastAPI**.

Factores decisivos

1. Python es ampliamente usado, teniendo varios frameworks populares, miles de librerías y documentación extensa sobre su uso. Además, todo el equipo ya es extremadamente familiar con el lenguaje.
2. La arquitectura demanda tanto una API REST como una API WebSocket; y FastAPI soporta ambas de manera nativa sin necesidad de agregar librerías externas ni configuración extra.
3. Es un framework pensado para la comunicación asíncrona, la cual nuestro sistema utiliza extensivamente.
4. Opciones como la validación automática de tipos de datos mediante Pydantic o la generación de documentación OpenAPI son puntos a favor que pueden hacer más ameno el desarrollo.

Alternativas

- **Flask**: Es simple y muy conocido, pero no fue diseñado para ser asíncrono. El soporte de WebSockets requiere dependencias adicionales como `flask-socketio`,

lo que introduce complejidad innecesaria y una integración forzada de la comunicación en tiempo real.

- **Django + Django Channels:** Ofrece WebSockets a través de Channels y es un framework muy completo, pero está diseñado alrededor de su propio gestor de datos relacionales y sistema de autenticación, ninguno de los cuales se usaría aquí. Es excesivo para nuestra escala.

Pros y Contras de Python + FastAPI

Pros:

- Soporte nativo de REST y WebSockets.
- Alto rendimiento en comunicaciones asíncronas.
- Validación y serialización automática de datos con Pydantic.
- Sistema de dependencias **Depends** muy útil, especialmente para la autenticación.

Contras:

- Al ser más reciente, hay menos documentación y estándares que los disponibles para Flask o Django.

ADR-014

Título

Selección de un cliente PostgreSQL para Python, y una forma sencilla de gestionarlo.

Fecha

11/04/2026

Estado

Tomada

Contexto

Las interacciones con la base de datos relacional deben hacerse desde el componente **Persistencia**, que forma parte del backend y por lo tanto será escrito en Python. Es entonces necesario elegir una herramienta y driver capaces de mapear datos relacionales a Python, que a su vez deben poder funcionar de manera asíncrona para no bloquear el backend mientras se espera una respuesta de la base de datos..

Decisión tomada

Basar las interacciones con la base de datos relacional en SQLAlchemy Core (No ORM) con psycopg3 de driver.

Factores decisivos

1. SQLAlchemy Core permite escribir consultas SQL de manera explícita y directa, lo que es apropiado para un sistema donde las queries pueden ser específicas y son conocidas de antemano. No se necesita la abstracción completa de un ORM.
2. Psycopg3 sigue el estándar DB-API 2.0 de Python, que es la interfaz que se esperaría encontrar en un cliente de base de datos de Python. Esto significa que el código se lee y se escribe de forma natural, y familiarizarse con este estándar es valioso y útil para el equipo.
3. La integración con SQLAlchemy es directa, simplemente se especifica psycopg como driver en **connection string** y funciona.

Alternativas

- **SQLAlchemy ORM:** Añade una abstracción al mapear entidades a clases de Python. Para este sistema, donde las consultas son relativamente directas y el equipo tiene control total del esquema, dicha abstracción agrega complejidad sin beneficios claros.

- **psycopg3 solo:** Da control total sobre las queries. Sin embargo, obliga a gestionar manualmente la conexión y otros detalles que SQLAlchemy resuelve y simplifica.
- **SQLAlchemy Core con asyncpg:** Asyncpg es el driver asíncrono más rápido para PostgreSQL en Python; sin embargo, tiene una API no estándar que no sigue DB-API 2.0, y en su lugar utiliza su propia API ligeramente más compleja, e inconveniente. Para la escala y el contexto de este proyecto, la diferencia de rendimiento que nos daría es despreciable.

Pros y Contras de SQLAlchemy Core con psycopg3

Pros:

- SQLAlchemy simplifica y abstrae enormemente todo el proceso y mantenimiento de la conexión a la base de datos, así como las interacciones.
- SQLAlchemy Core permite formular las queries con comandos de SQL, facilitando la generación de las consultas y mejorando la experiencia de desarrollo.
- Psycopg3 ofrece cliente sync y async en el mismo paquete, con la misma interfaz.
- API familiar y estándar (DB-API 2.0).

Contras:

- Levemente más lento que asyncpg, diferencia que en la práctica no tiene impacto para este sistema.



ADR-015

Título

Selección de una librería de autenticación y una librería de hashing de contraseñas para el componente **Autenticación** del backend.

Fecha

11/04/2026

Estado

Tomada

Contexto

Dado que la arquitectura define un componente de autenticación, utilizado tanto por la API REST como por la API WebSockets, es necesario decidir cómo se generarán y validarán las sesiones de usuario, y cómo se almacenarán de forma segura las contraseñas en la base de datos PostgreSQL. El sistema no requiere integración con proveedores externos (como Google u otros), por lo que se busca una solución de autenticación simple pero segura.

Decisión tomada

Se utilizará **PyJWT** para la generación y validación de tokens de sesión bajo el estándar JWT (JSON Web Tokens), y **passlib** con el backend **bcrypt** para el hashing seguro de contraseñas al momento de registrarlas y verificarlas.

Factores decisivos

1. El sistema solo requiere autenticación propia mediante usuario y contraseña con sesiones stateless basadas en tokens; no se necesita ninguna funcionalidad de OAuth, ni login con terceros, por lo que una solución liviana es la más adecuada.
2. FastAPI provee un sistema de inyección de dependencias (**Depends**) que permite construir un verificador de tokens JWT reutilizable con muy poco código, haciendo que PyJWT sea suficiente para cubrir la necesidad sin frameworks más complejos.
3. bcrypt es el algoritmo estándar de la industria para almacenar contraseñas hashadas de forma segura; passlib ofrece una interfaz de alto nivel sobre bcrypt que simplifica su uso correcto.

Alternativas

- **python-jose**: librería similar a PyJWT con soporte para un mayor número de algoritmos criptográficos (RSA, EC). Fue la opción recomendada en versiones

anteriores de la documentación oficial de FastAPI, pero hoy en día PyJWT resulta más moderno y mejor mantenido. Además, dado que el sistema solo requiere el algoritmo HS256, el soporte adicional de algoritmos no representa un beneficio real.

- **Authlib:** framework completo de OAuth 2.0 y OpenID Connect para Python. Sería la elección correcta si el sistema necesitara asociar el login con proveedores externos o exponer endpoints OAuth propios. Para un sistema de autenticación interna y básica como el de este proyecto, introduce una complejidad y una cantidad de configuración muy superiores a lo necesario.

Pros y Contras de PyJWT y passlib con bcrypt

Pros:

- PyJWT es extremadamente liviano y hace exactamente una cosa: codificar y decodificar tokens JWT, sin efectos secundarios ni abstracciones innecesarias.
- La combinación se integra de forma natural con el sistema `Depends()` de FastAPI.
- passlib abstrae el uso correcto de bcrypt, reduciendo la probabilidad de errores de implementación.

Contras:

- Al no utilizar un framework de autenticación completo, funcionalidades como la revocación de tokens activos o el manejo de refresh tokens van a obviarse para este proyecto, y deberían implementarse manualmente si se quisieran en el futuro.

Consecuencias

El componente será responsable de exponer al menos dos operaciones internas: generación de un token JWT al momento del login, y validación del token en cada request. Ambas APIs (REST y WebSockets) usarán esta lógica mediante el sistema de dependencias de FastAPI.

Las contraseñas nunca se almacenarán en texto plano; únicamente el hash bcrypt será persistido en la base de datos.

ADR-016

Título

[Título breve y claro de la decisión arquitectónica tomada]

Fecha

[DD/MM/AAAA]

Estado

[Por decidirse | Tomada | Obsoleta]

Contexto

[Descripción clara y completa del problema o situación que motivó la decisión.]

Decisión tomada

[Descripción concreta de lo que se decidió.]

Factores decisivos

- [Motivo principal 1]
- [Motivo principal 2]
- [Motivo principal 3]

Alternativas

- [Alternativa 1: descripción y motivo de descarte]
- [Alternativa 2: descripción y motivo de descarte]

Pros y Contras de [nombre de la decisión]

Pros:

- [Ventaja 1]
- [Ventaja 2]

Contras:

- [Desventaja 1]
- [Desventaja 2]

Consecuencias (opcional)

[Descripción de los efectos prácticos que tendrá esta decisión.]

Comentarios (opcional)

[Cualquier detalle adicional o nota.]

ADR-017

Título

[Título breve y claro de la decisión arquitectónica tomada]

Fecha

[DD/MM/AAAA]

Estado

[Por decidirse | Tomada | Obsoleta]

Contexto

[Descripción clara y completa del problema o situación que motivó la decisión.]

Decisión tomada

[Descripción concreta de lo que se decidió.]

Factores decisivos

- [Motivo principal 1]
- [Motivo principal 2]
- [Motivo principal 3]

Alternativas

- [Alternativa 1: descripción y motivo de descarte]
- [Alternativa 2: descripción y motivo de descarte]

Pros y Contras de [nombre de la decisión]

Pros:

- [Ventaja 1]
- [Ventaja 2]

Contras:

- [Desventaja 1]
- [Desventaja 2]

Consecuencias (opcional)

[Descripción de los efectos prácticos que tendrá esta decisión.]

Comentarios (opcional)

[Cualquier detalle adicional o nota.]

ADR-018

Título

[Título breve y claro de la decisión arquitectónica tomada]

Fecha

[DD/MM/AAAA]

Estado

[Por decidirse | Tomada | Obsoleta]

Contexto

[Descripción clara y completa del problema o situación que motivó la decisión.]

Decisión tomada

[Descripción concreta de lo que se decidió.]

Factores decisivos

- [Motivo principal 1]
- [Motivo principal 2]
- [Motivo principal 3]

Alternativas

- [Alternativa 1: descripción y motivo de descarte]
- [Alternativa 2: descripción y motivo de descarte]

Pros y Contras de [nombre de la decisión]

Pros:

- [Ventaja 1]
- [Ventaja 2]

Contras:

- [Desventaja 1]
- [Desventaja 2]

Consecuencias (opcional)

[Descripción de los efectos prácticos que tendrá esta decisión.]

Comentarios (opcional)

[Cualquier detalle adicional o nota.]

ADR-019

Título

[Título breve y claro de la decisión arquitectónica tomada]

Fecha

[DD/MM/AAAA]

Estado

[Por decidirse | Tomada | Obsoleta]

Contexto

[Descripción clara y completa del problema o situación que motivó la decisión.]

Decisión tomada

[Descripción concreta de lo que se decidió.]

Factores decisivos

- [Motivo principal 1]
- [Motivo principal 2]
- [Motivo principal 3]

Alternativas

- [Alternativa 1: descripción y motivo de descarte]
- [Alternativa 2: descripción y motivo de descarte]

Pros y Contras de [nombre de la decisión]

Pros:

- [Ventaja 1]
- [Ventaja 2]

Contras:

- [Desventaja 1]
- [Desventaja 2]

Consecuencias (opcional)

[Descripción de los efectos prácticos que tendrá esta decisión.]

Comentarios (opcional)

[Cualquier detalle adicional o nota.]

ADR-020

Título

[Título breve y claro de la decisión arquitectónica tomada]

Fecha

[DD/MM/AAAA]

Estado

[Por decidirse | Tomada | Obsoleta]

Contexto

[Descripción clara y completa del problema o situación que motivó la decisión.]

Decisión tomada

[Descripción concreta de lo que se decidió.]

Factores decisivos

- [Motivo principal 1]
- [Motivo principal 2]
- [Motivo principal 3]

Alternativas

- [Alternativa 1: descripción y motivo de descarte]
- [Alternativa 2: descripción y motivo de descarte]

Pros y Contras de [nombre de la decisión]

Pros:

- [Ventaja 1]
- [Ventaja 2]

Contras:

- [Desventaja 1]
- [Desventaja 2]

Consecuencias (opcional)

[Descripción de los efectos prácticos que tendrá esta decisión.]

Comentarios (opcional)

[Cualquier detalle adicional o nota.]
